



Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Case Study: Joining the Tables

WEEK 2 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Three clean tables, and one book to build

What yesterday found

Table	Rows	Columns	One row is
positions	200	6	one bond held
issuers	150	5	one issuer
ratings	18	4	one rating

489,000,000

of face held in total

0

missing
values or
duplicates

2

keys tested:
ticker and
rating

The report to the desk: the three tables are clean and they fit together.

Your job today

Build

one table, the book: one row per bond, with every attribute the desk needs

Prove

that nothing was lost and nothing was duplicated on the way

Answer

the desk's nine questions from that one table

Two joins, and a check around each

positions merge on ticker issuers → **step_one**

Gives each bond its issuer's name, sector, country, and rating.

step_one merge on rating ratings → **book**

Gives each bond its score, its bucket, and its grade.

The check

Written down before any join:

200 rows

489,000,000 of total face

After each join, both must be unchanged.

```
step_one = pd.merge(positions, issuers, on='ticker', how='left', validate='many_to_one')
book = pd.merge(step_one, ratings, on='rating', how='left', validate='many_to_one')
# how='left' keeps every row of the left table. validate= stops the merge if a key is listed twice in the lookup.

assert book.shape[0] == rows_before, 'the second merge changed the row count'
assert book['par_held'].sum() == face_before, 'the second merge changed the total face'
# assert stops the notebook with its message if a condition is False
```

The same join in Excel

a lookup, filled down the sheet

=VLOOKUP(B2, issuers!A:E, 3, FALSE)

the sector: column 3 of A to E

=XLOOKUP(B2, issuers!A:A, issuers!C:C)

the sector, in current Excel

The book: 200 rows and 13 columns

200

rows, after both joins

13

columns: 6, plus 4, plus 3

489,000,000

of face, unchanged

0

gaps in the new columns

```
book.iloc[0]
cusip          99080CAE8
ticker         BEZE
coupon         7.0
maturity       2032-06-18
price          106.727
par_held       5000000
issuer         Bezane Finance
sector         Financials
country        US
rating         BBB-
score          10
bucket         BBB
grade          Investment Grade
```

The check after the second join. Both assert lines passed in silence.

```
Rows before: 200  after: 200
Face before: 489,000,000  after: 489,000,000
Rows unchanged: True
Face unchanged: True
```

Before the second join

```
step_one.groupby('grade')
KeyError: 'grade'. The column is not
in the table yet.
```



After it

```
book.groupby('grade')
It runs. The grade came from the
ratings table.
```

An outer join keeps every key from both tables

```
everything = pd.merge(positions, issuers, on='ticker',
                      how='outer', indicator=True)
everything['_merge'].value_counts()

both          200
right_only    35
left_only      0
```

200 both: a position that found its issuer

35

235 rows. 35 right_only: an issuer with no position

These 35 rows are not errors in the data. They are issuers the desk does not hold, and they do not belong in a table with one row per bond held.

What a right_only row looks like

ticker	issuer	cusip	par_held
BEZY	Bezolvey Chemicals	NaN	NaN
BRUN	Bruxiskan Pipeline	NaN	NaN

A report built on this table by mistake

489,000,000

of face. The face check passes: sum() skips the missing values.

235

rows, against 200. The row check fails. As an assert, it stops the run:

AssertionError: expected 200 rows and found 235

One check passes and one fails. Always run both.

A repeated key adds rows, with no error

Four positions, and a lookup that lists BEZL twice

```
doubled = pd.merge(small, lookup_twice, on='ticker', how='left')
```

	cusip	ticker	par_held	sector
0	99080CAE8	BEZE	5000000	Financials
1	99049XAE2	BEZH	250000	Industrials
2	99039NAC0	BEZL	1500000	Transportation
3	99039NAC0	BEZL	1500000	Transportation
4	99039NAG1	BEZL	500000	Transportation
5	99039NAG1	BEZL	500000	Transportation

Face before: 7250000 after: 9250000

6 rows

from 4: each BEZL bond
matched twice

9,250,000

of face, from 7,250,000

The same lookup sheet in Excel

VLOOKUP and XLOOKUP return the first match and stop. No row is added, and the repeat is not reported. pandas returns every match.

Join written as	Rows	Face	Caught by
left, each key once in the lookup	200	489,000,000	nothing to catch
outer	235	489,000,000	the row check
left, a key repeated in the lookup	more than the left table	too high	both checks, and validate= before them
inner, a key missing from the lookup	fewer than the left table	too low	both checks

Preparing the book: five new columns

Column	pandas	Excel, row 2
Value	<code>book['par_held'] * book['price'] / 100</code>	<code>=F2 * E2 / 100</code>
Coupon income	<code>book['par_held'] * book['coupon'] / 100</code>	<code>=F2 * C2 / 100</code>
Years to maturity	<code>(book['maturity'] - as_of).dt.days / 365.25</code>	<code>=(D2 - DATE(2026, 9, 30)) / 365.25</code>
Weight	<code>book['value'] / total_value * 100</code>	<code>=N2 / SUM(\$N\$2:\$N\$201) * 100</code>

Value at the clean price

face x clean price / 100

The first bond: 5,000,000 of face at 106.727 is 5,336,350. This is before accrued interest, so it is not a full market value.

A date, then a bucket

`pd.to_datetime()` `pd.cut()`

The maturity arrived as text. As a date it gives the years left, and `pd.cut()` sorts those into 0 to 3, 3 to 7, 7 to 15, and 15 and over.

Helper columns in Excel

`$N$2:$N$201`

Each new column is a formula filled down the sheet. The dollar signs hold the total in place. In pandas the total is one stored number.

The desk's questions: sector, grade, rating

Bonds and value by sector, largest first

sector	bonds	value	pct_of_value
Materials	26	70,527,440	14.60
Financials	26	67,067,357.5	13.88
Consumer	26	55,773,490	11.55
Telecom	21	54,953,580	11.38
Industrials	20	51,658,930	10.69
Transportation	20	48,159,510	9.97
Health Care	20	38,268,482.5	7.92
Energy	17	38,258,305	7.92
Technology	12	29,826,430	6.17
Utilities	12	28,530,705	5.91

In Excel: a pivot table with sector in Rows, and Count of cusip and Sum of value in Values.

The two grades: share of bonds and share of value

grade	bonds	pct_of_bonds	pct_of_value
High Yield	72	36.0	34.45
Investment Grade	128	64.0	65.55

In Excel, with the grade in column M and the value in column N

=COUNTIF(M:M, "High Yield") 72 bonds

=SUMIF(M:M, "High Yield", N:N) 166,398,102.5

Percent of value by rating bucket, sorted on the score

AAA	AA	A	BBB	BB	B	CCC
1.68	2.68	30.38	30.82	23.09	8.83	2.53

The desk's questions: averages, issuers, maturity

Average coupon, in percent, and average rating score

Average	coupon	score
Simple	5.9375	9.36
Value-weighted	5.9403	9.25

In Excel, with the coupon in column C and the value in column N

=AVERAGE(C2:C201)

=SUMPRODUCT(C2:C201, N2:N201) / SUM(N2:N201)

483,024,230

of value at the clean price, before accrued interest. The answers describe the data and are not a view.

Value by sector and grade: three rows of the pivot table

sector	High Yield	Investment Grade
Materials	38,003,020	32,524,420
Telecom	27,344,940	27,608,640
All	166,398,102.5	316,626,127.5

In Excel: the same pivot table, with grade dragged into Columns.

20.67%

ten largest of 115 issuers

46.61%

in the 7 to 15 year bucket

113

bonds priced below par

28

bonds mature within 3 years

Save the book, then read it back

To a CSV file, and back

```
book.to_csv('desk_book_joined.csv', index=False)
read_back = pd.read_csv('desk_book_joined.csv',
                        parse_dates=['maturity'])
```

```
Shape saved: (200, 18)  read back: (200, 18)
Value saved: 483,024,230  read back: 483,024,230
Face saved: 489,000,000  read back: 489,000,000
```

To an Excel workbook with two sheets, and back

```
with pd.ExcelWriter('desk_book_joined.xlsx') as writer:
    book.to_excel(writer, sheet_name='book', index=False)
    by_sector.to_excel(writer, sheet_name='by_sector')
from_excel = pd.read_excel('desk_book_joined.xlsx',
                           sheet_name='book', dtype={'cusip': str})
```

```
Shape saved: (200, 18)  read back: (200, 18)
Value saved: 483,024,230  read back: 483,024,230
```

Column	In the book	From the CSV	From the workbook
maturity	datetime64[us]	datetime64[us]	datetime64[us]
maturity_bucket	category	str	str

Both files return the bucket as text, so run `pd.cut()` again. Older versions of pandas show object where this says str.

What each file keeps

A CSV holds characters only, so `parse_dates` asks for the date. A workbook stores a date as a date.

The check, once more

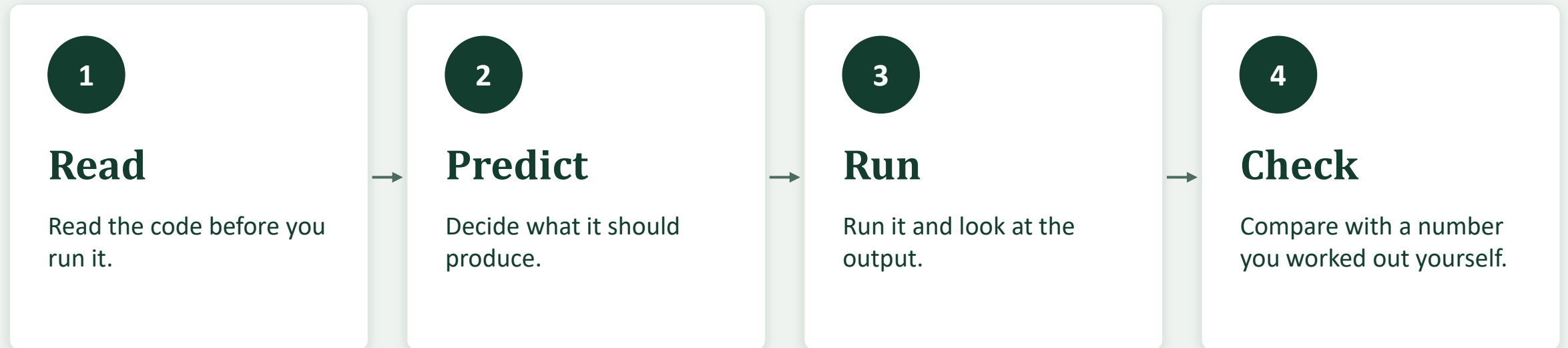
Shape, value, and face match after each read, and every assert line passes.

The same in Excel

File, Save As: CSV for one table, or Excel Workbook with a book tab and a by_sector tab.

The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.



Code that runs is not the same as code that is right.

What you can do now

- Join tables on a key with `pd.merge()`, and `how='left'` to keep every row
- Check the rows and a total around every join, with `validate=` and `assert`
- Read `_merge` from an outer join to see what did not match
- Prepare new columns, with the unit in the name
- Answer by group with `groupby()`, `agg()`, and `pivot_table()`

Week 3 Charts and exploratory data analysis: the same book, and the same questions, as pictures